Funciones

Martín René Vilugrón
mrvilugron@unrn.edu.ar

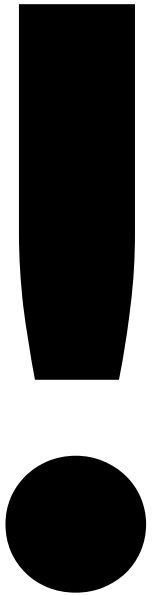
UNRN
Universidad Nacional
de Río Negro

r33

05 / 2026



Una pieza fundamental en programación



Como una oficina

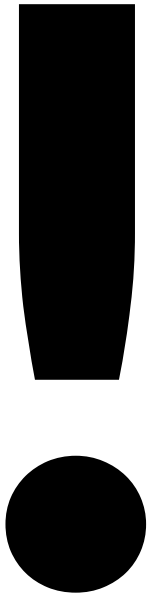


UNRN Universidad Nacional de Río Negro


salida
oficina
entrada
entrada



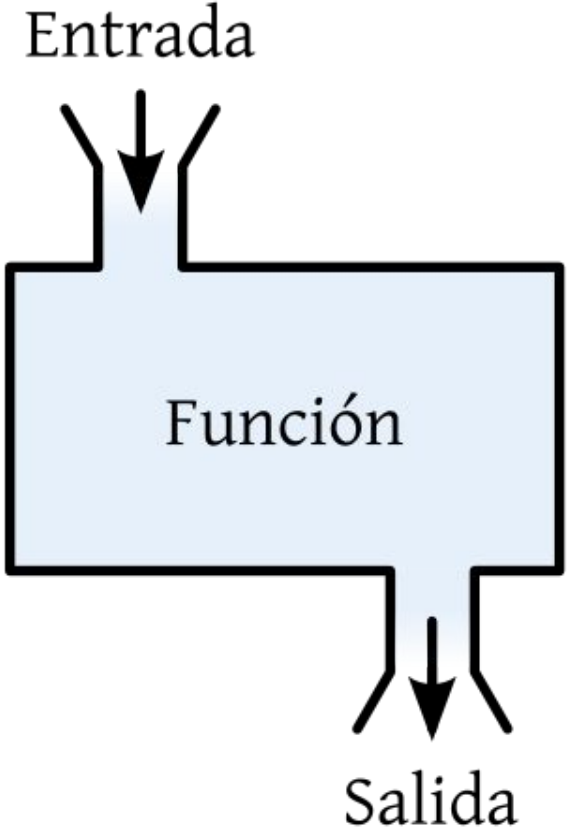
Una ~~*pieza*~~ abstracción fundamental en programación



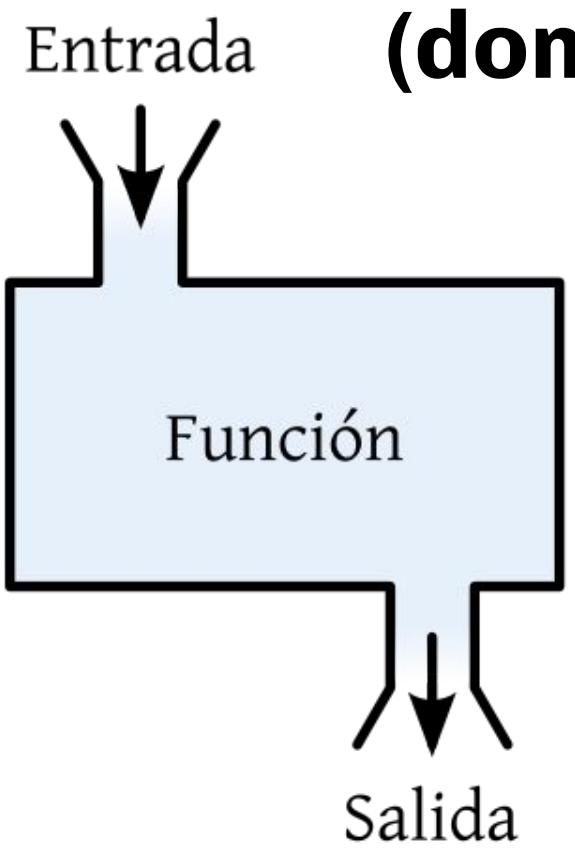
Concepto

UNRN Universidad Nacional de Río Negro

Definición conceptual de una función



¿Cómo es una función?



(dominio)

(imagen)


salida
función
entrada
entrada

salida **entrada**

```
tipo_retorno identificador (lista_argumentos )
{
  //bloque de instrucciones (cuerpo)
}
```

La firma de la función

En donde cada ítem separado por coma de `lista_argumentos` es

`tipo_argumento` `identificador`

de salida
tipo
solo uno*

```
return variable;
```

funciona igual* que Python

Y devuelve el control a quien la llamó



```
int suma(int op1, int op2)
{
    int temp = op1 + op2;
    return temp;
}
```

Los identificadores deben ser descriptivos**

```
int suma(int sumando, int sumador)
{
    int sumado = sumando + sumador;
    return sumado;
}
```

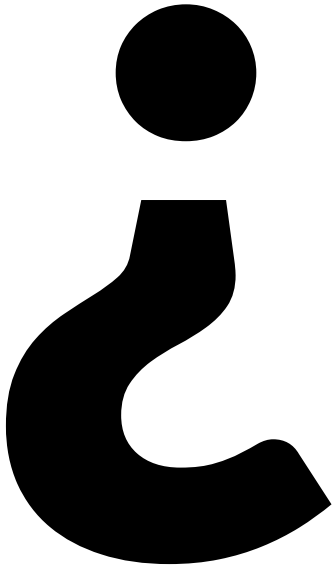
Una función en sí es un

R-VALUE

Formas de llamada a función

```
int resultado = suma(10, 45);
int plus = suma(suma(10,20), 10);
int otro = suma(resultado, 10);
```

Con clases de argumentos diferentes



¿Puede no tener salida?



si, y es llamada

procedimiento

o

Cuando no devuelve nada, se indica con la palabra reservada

void

```
void mi_funcion()
{
    bloque;
}
```

Un ejemplo de procedimiento (y sin argumentos también)

```
void mi_funcion()
{
  return;
}
```

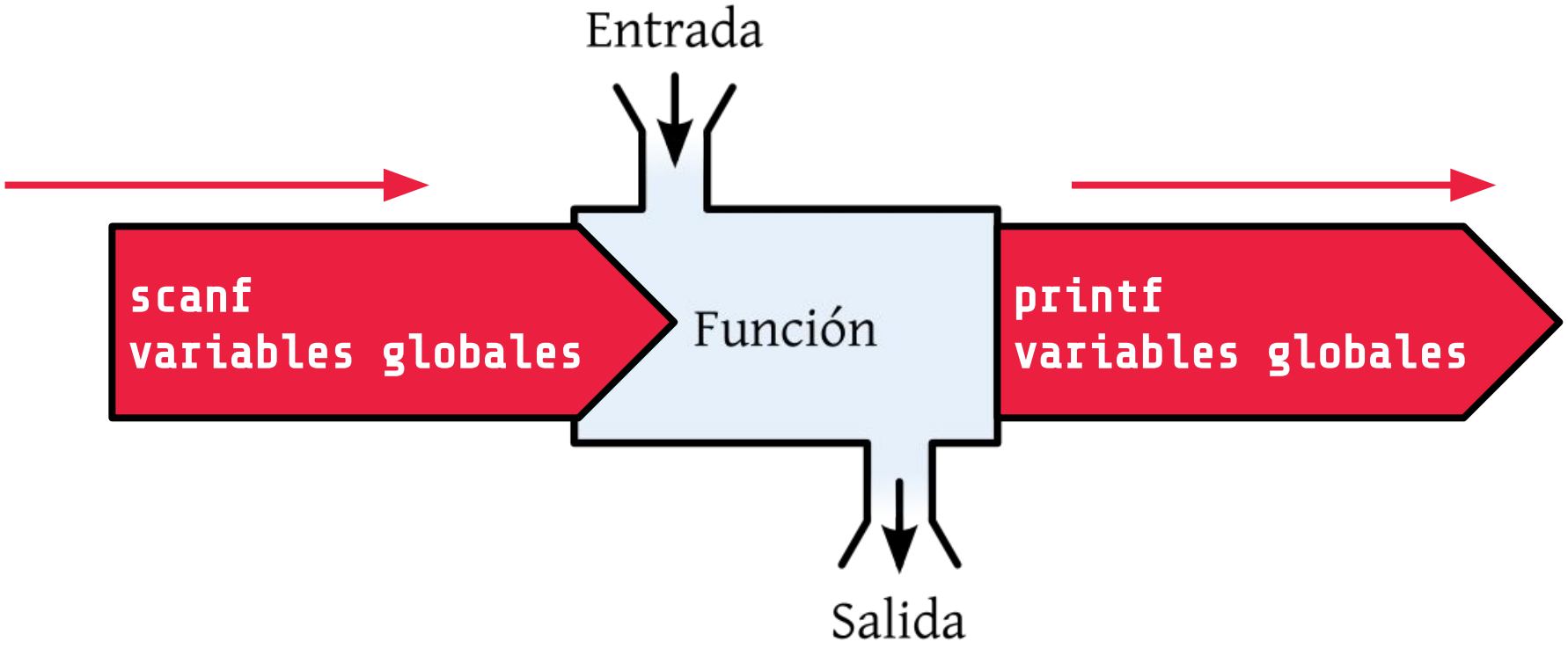
Un procedimiento puede usar `return`

Un ejemplo de procedimiento

```
void muestra_mensaje()
{
    printf("Hola Mundo!\n");
}
```

¿Para qué queremos algo que no da resultados?

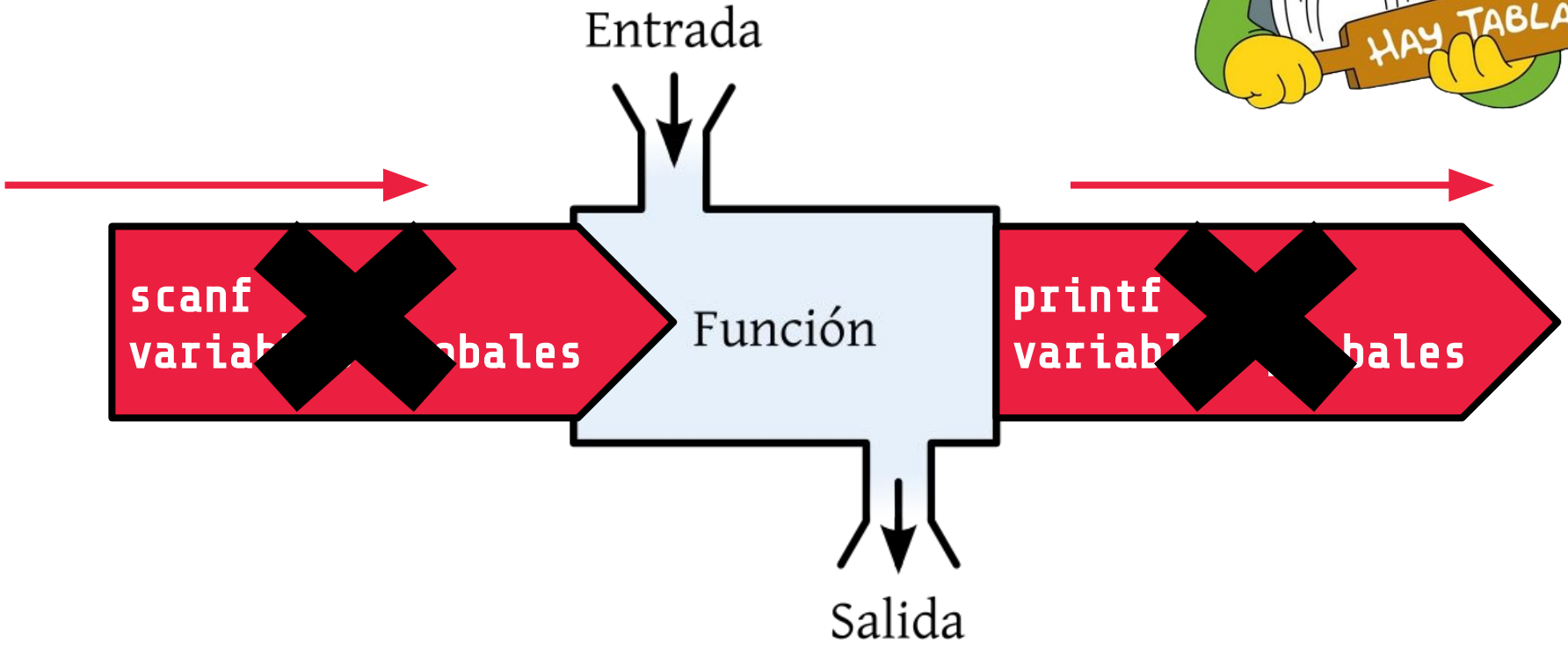
Otras formas de pasar información a una función



{ Son llamados efectos secundarios }

¡Y no están permitidos!





Función pura

Una **función pura** es una que siempre da el mismo resultado a partir de los mismos argumentos.

Y no tiene efectos secundarios

Salvo para cosas como por ejemplo:

```
int leer_entero()
{
    int valor = 0;
    scanf("pasame un valor entero %d", &valor);
    return valor;
}

void mostrar_temperatura(double celcius)...
```

En donde el efecto secundario es el proposito

Las funciones *no van* con `printf` o `scanf`

a no ser que ese sea su propósito

Scopes I

¿Que es una variable global?

UNRN Universidad Nacional de Río Negro

¿Dónde se declaran?

```
int variable_global = 10;

int main() {
  printf("La variable global es %d\n", variable_global);

  return 0;
}

void procedimiento(){
  printf("La variable_global existe acá %d\n", variable_global);
}
```

Global

Las variables declaradas fuera de cualquier función tienen alcance global. Esto significa que son accesibles desde cualquier parte del programa.

¡A la terminal!



```c
int main() {
  int variable_local = 20;
  printf("La variable local vale %d\n", variable_local);
  return 0;
}

void procedimiento(){
  printf("La variable_local no existe acá %d\n", variable_local);
}
```

Local

Las que venimos utilizando;
Las variables declaradas dentro de una función tienen alcance local. Esto significa que solo son accesibles desde dentro de la función en la que se declaran.

¡A la terminal!



Las llaves pueden ir ‘sueltas’ :-)

```
int main()
{
  {
    int variable_bloque = 30;

    // La variable variable_bloque solo es accesible desde dentro del bloque
    printf("El valor de la variable de bloque es %d\n", variable_bloque);
  }

  // La variable variable_bloque no es accesible fuera del bloque
  printf("El valor de la variable de bloque es %d\n", variable_bloque);

  return 0;
}
```

Pero también incluye las llaves de las estructuras de control

Bloques { }

Las variables declaradas dentro de un bloque tienen alcance de bloque. Esto significa que solo son accesibles desde dentro del bloque en el que se declaran.

¡A la terminal!

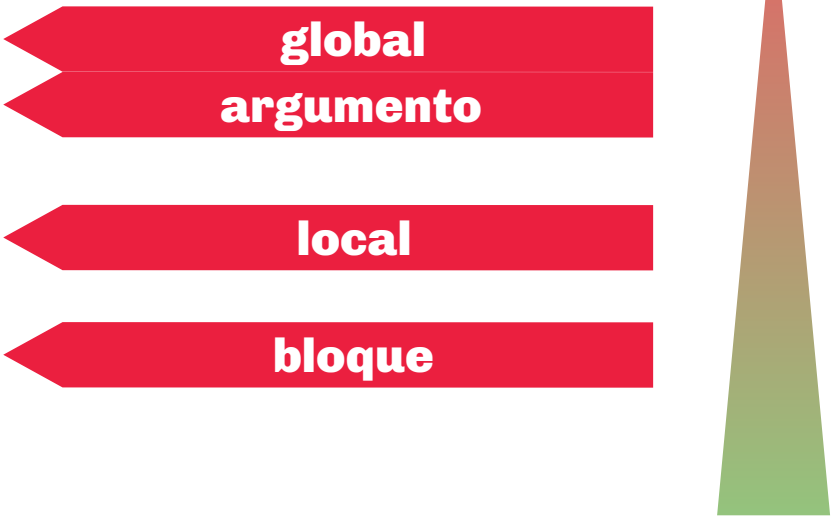


Indica en dónde es visible y accesible

```
int uno = 0x0FE0;
void funcion(int dos)
{
  int tres = 0xBEEF;
  {
    int cuatro = 0xCAFE;
  }
}
```

Se van ‘yendo’ con cada }

```
int uno = 0x0FE0;
void funcion(int dos)
{
  int tres = 0xBEEF;
  {
    int cuatro = 0xCAFE;
  } // cuatro, afuera
} // dos y tres, afuera
```



¿Y ahora?

```
int uno = 0x0FE0;
int main()
{
  int uno = 0xBEEF;
  {
    int uno = 0xCAFE;
  }
  return uno;
}
```

```
int otra(int uno)
{
  int uno = 0xFECA;
  {
    int dos = 0xFEFE;
  }
  return uno;
}
```

Sin usar variables globales

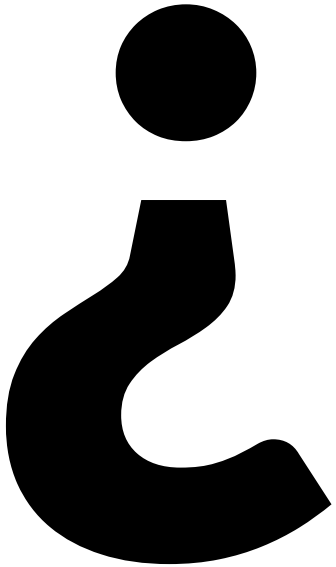

"We don't do that here"


¿Preguntas?

Encadenando funciones



UNRN Universidad Nacional de Río Negro



una función puede llamar a otra




¡Sí!

¿El orden de las funciones es importante?

*no


¡Sí!

```
void funcion_1()
{
    funcion_2();
}

void funcion_2()
{
    funcion_1();
}
```

Prototipos de función

UNRN Universidad Nacional de Río Negro

Es la declaración de función con argumentos y retorno pero sin cuerpo.

```
void mi_funcion();
```

El ‘prototipo’ no es necesario en este caso

```
void saludo()
{
    printf(“Hola Mundo”);
}

int main()
{
    saludo();
    return 0;
}
```

Acá se implementa antes de usarse

Se usa pero ya está implementada

Un ejemplo más completo

```
void saludo();

int main()
{
    saludo();
    return 0;
}

void saludo()
{
    printf("Hola Mundo");
}
```

Es necesario

Porque se usa antes de su implementación

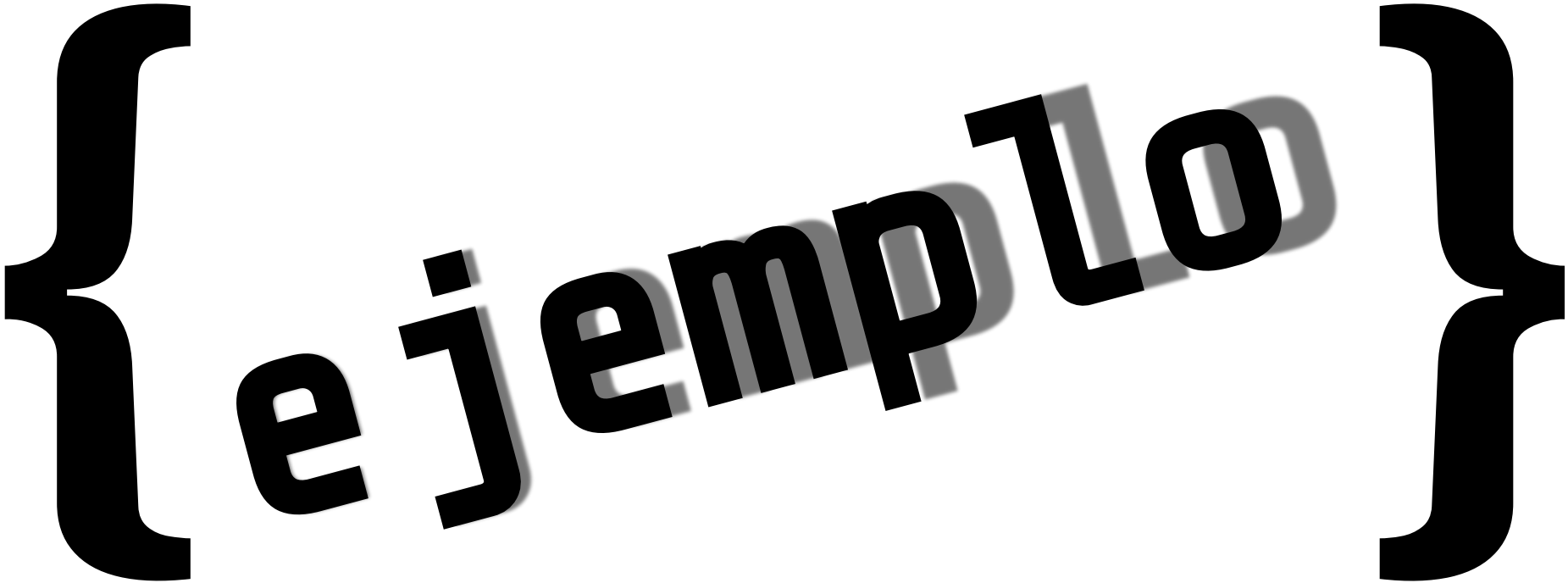
¡A la terminal!




¿Preguntas?

Pasaje de argumentos

UNRN Universidad Nacional de Río Negro


{ejemplo}

Ejemplo I

```c
#include <stdio.h>

int incremento(int variable)
{
  variable = variable+ 1;
}

int main()
{
  int variable = 10;
  variable = incremento(variable);
  printf("variable: %d\n", variable);
  return 0;
}
```

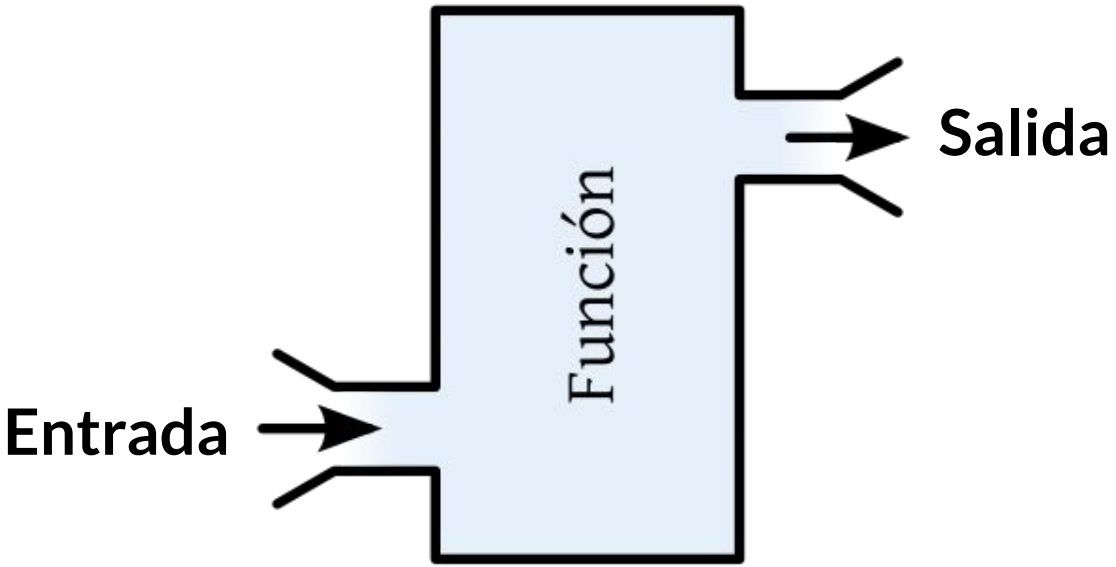
Ejemplo II

```c
#include <stdio.h>

int incremento(int variable)
{
  variable = variable+ 1;
  return variable;
}

int main()
{
  int variable = 10;
  incremento(variable);
  printf("variable: %d\n", variable);
  return 0;
}
```

—



Esto se “mantiene”, si no hay salida, no hay cambios afuera.

¡no hay cambios!

¡es lo importante!

Lo que ‘*entra*’ es una copia del valor de la variable

Lo que pasa en la función queda en la función

```
void incremento(int valor)
{
    valor = valor + 1;
}
```

Lo que pasa en la función queda en la función

```
int incremento(int valor)
{
    return valor + 1;
}
```

Ejemplo III - Consola en vivo

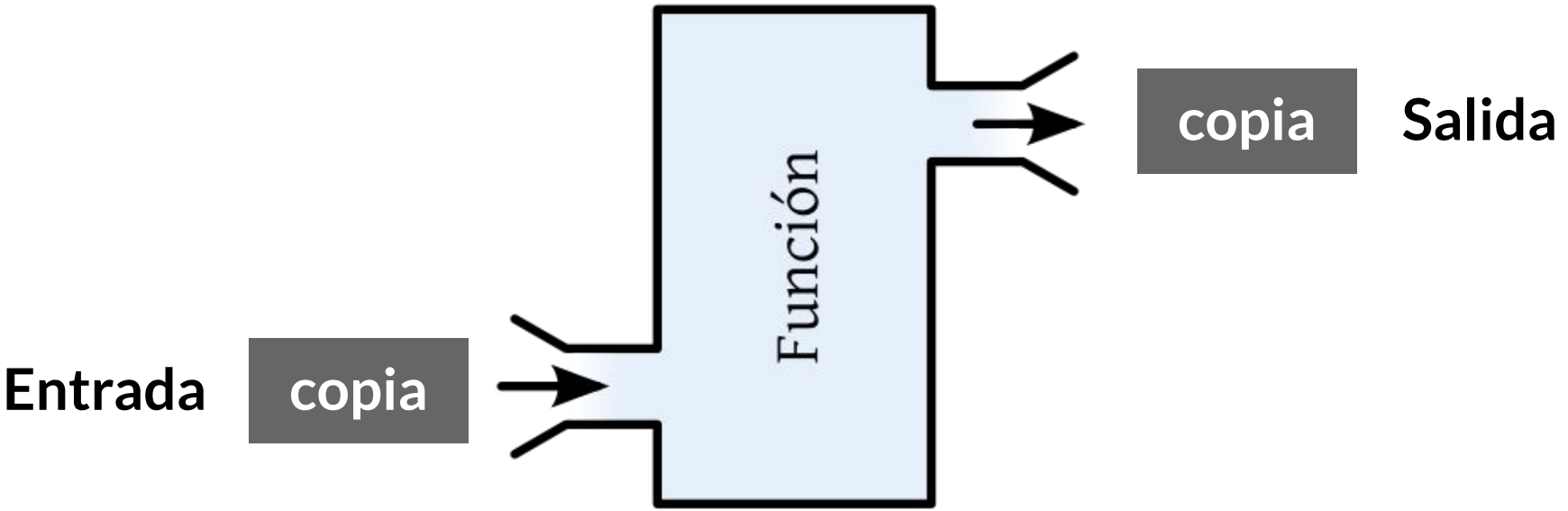
```c
#include <stdio.h>

int incremento(int valor)
{
    return valor + 1;
}

int main()
{
    int variable = 10;
    variable = incremento(variable);
    printf("variable: %d\n", variable);
    return 0;
}
```

¿Y ahora?

—



Si no guardamos la salida, no hay cambios

La llamada a la función es un r-value

Que es necesario guardar para que ‘tenga sentido’

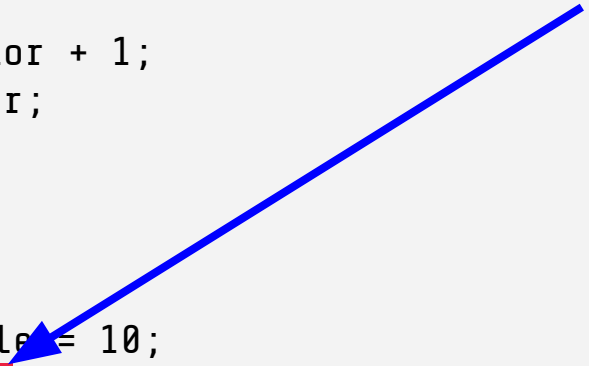
Ejemplo IV - Consola en vivo

```
 * Ejemplo pasaje de argumentos
 */

#include <stdio.h>

int incremento(int valor)
{
    valor = valor + 1;
    return valor;
}

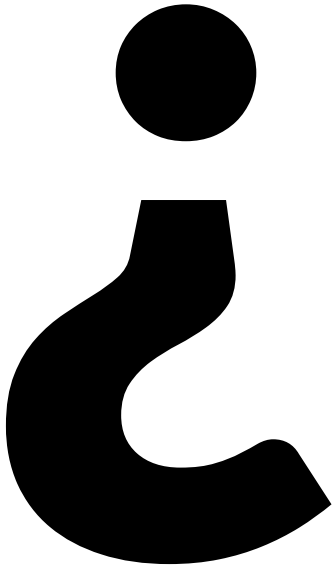
int main()
{
    int variable = 10;
    variable = incremento(variable);
    printf("variable: %d\n", variable);
    return 0;
}
```



Hay que guardar la salida

{ En C los valores siempre van por copia }


¿Preguntas?



Por qué existen las funciones



Inicialización y día

```
int main(void) {
    int dia, mes, anio;

    printf("Ingrese día (1-31): ");
    scanf("%d", &dia);
    while (dia < 1 || dia > 31) {
        printf("Valor inválido. Ingrese día (1-31): ");
        scanf("%d", &dia);
    }
```

Para el mes

```
printf("Ingrese mes (1-12): ");
scanf("%d", &mes);
while (mes < 1 || mes > 12) {
    printf("Valor inválido. Ingrese mes (1-12): ");
    scanf("%d", &mes);
}
```

Para el año

```
printf("Ingrese año (1900-2026): ");
scanf("%d", &anio);
while (anio < 1900 || anio > 2026) {
    printf("Valor inválido. Ingrese año (1900-2026): ");
    scanf("%d", &anio);
}
```

Resultado

```
    printf("Fecha cargada: %02d/%02d/%04d\n", dia, mes, anio);
    return 0;
}
```

Procedimiento para anio

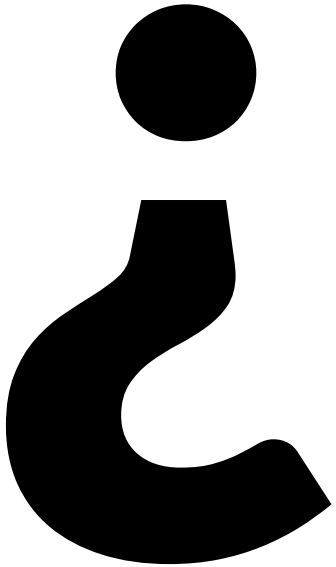
```
int leer_anio(void) {
    int anio;
    do {
        printf("Ingrese año (1900-2026): ");
        scanf("%d", &anio);
        if (anio < 1900 || anio > 2026) {
            printf("Error: año inválido.\n");
        }
    } while (anio < 1900 || anio > 2026);
    return anio;
}
```

Procedimiento para mes

```
int leer_mes(void) {
    int mes;
    do {
        printf("Ingrese mes (1-12): ");
        scanf("%d", &mes);
        if (mes < 1 || mes > 12) {
            printf("Error: mes inválido.\n");
        }
    } while (mes < 1 || mes > 12);
    return mes;
}
```

Procedimiento para día

```
int leer_dia(void) {
    int dia;
    do {
        printf("Ingrese día (1-31): ");
        scanf("%d", &dia);
        if (dia < 1 || dia > 31) {
            printf("Error: día inválido.\n");
        }
    } while (dia < 1 || dia > 31);
    return dia;
}
```



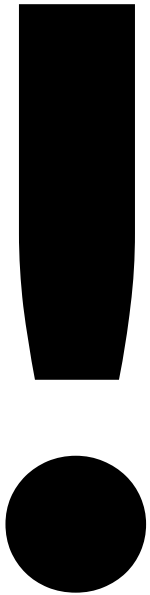
¿¿No hay nada sospechoso acá??



¿Qué es lo único que realmente cambia?



¡Son los 3 *casi* ¡iguales!



Y en lugar de copiar y pegar...

```
int leer_en_rango(int min, int max) {
    int valor;
    do {
        printf("Ingrese un valor entre %d y %d: ", min, max);
        scanf("%d", &valor);
        if (valor < min || valor > max) {
            printf("Error: fuera de rango.\n");
        }
    } while (valor < min || valor > max);
    return valor;
}
```

Si hay copypaste...

Hay algo para *revisar*


¿Preguntas?

¿Qué nos puede ayudar a esto?

Documentación

UNRN Universidad Nacional de Río Negro

Ejemplo de documentación completa

```
/*
 * Esta función se encarga de sumar dos numeros enteros
 *
 * @param izquierdo el primer operando
 * @param derecho el operando que será sumado al izquierdo
 * @return la suma de ambos parámetros
 */
int suma(int izquierdo, int derecho)
```

¿Por qué?

Ayuda a entender las responsabilidades de lo que programamos



escribiendo el código



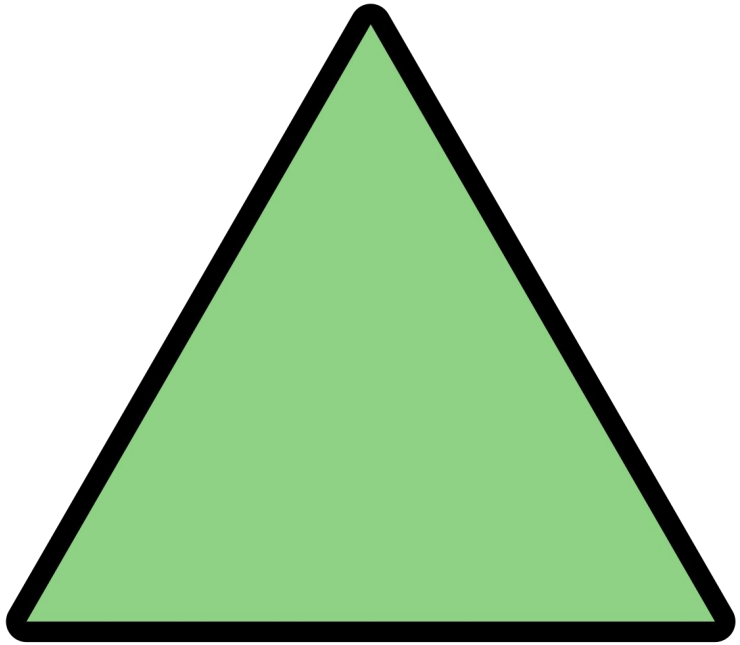
documentando el código

El triángulo de la documentación

o por qué el código por si mismo no lo es

Adaptado de:
https://sourceless.org/posts/the-documentation-triangle.html

qué



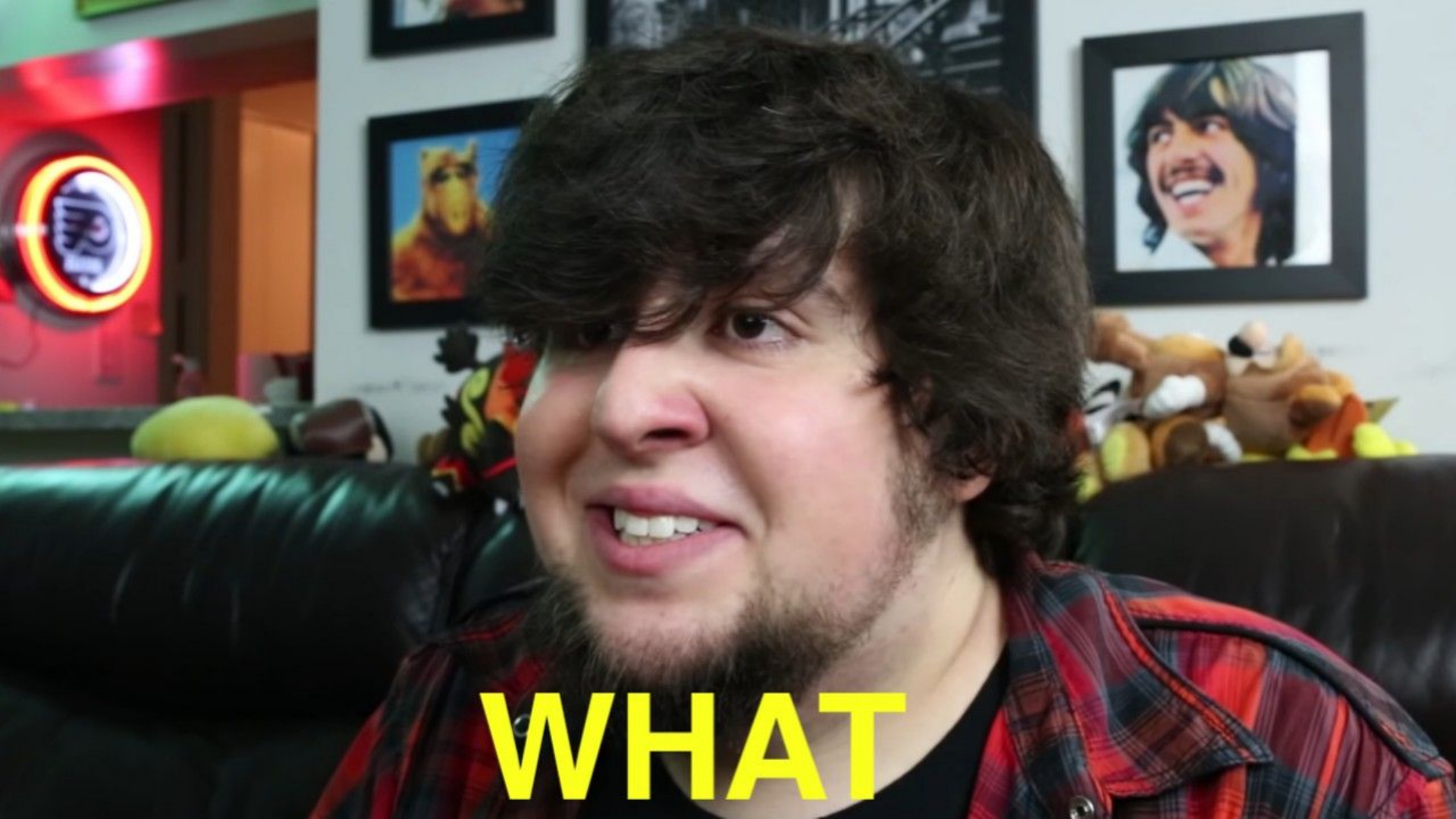
cómo

por qué

El objetivo de la documentación

qué

el código en sí


WHAT

por qué

Los comentarios


SUPER
中國紅 ZHON
SIEMPRE
PRECIOS LOCOS!!!

cómo

El contexto



UNRN

Tips documentación

0

Escriban la documentación antes de programar

Tips documentación

1

Mantener la prosa incluyendo el identificador del argumento y de la función

```
@arg numero que se suma
```

en lugar de

```
@arg numero numero que numero
```

Tips documentación

2

Sin repetir lo que está en el código
(el tipo, el identificador, por ejemplo)

```
@param veces que se divide
```

en lugar de

```
@param veces numero entero que...
```

Tips documentación

3

Cambien los identificadores si quedan en una mejor prosa al documentar

Tips documentación

4

Manténganla actualizada

Tips documentación

5

Indiquen qué valores son (y no) válidos (Divisor en 0 en división)

Usen debajo de `@returns`, `@postcondicion`

Tips documentación

6

Describan la salida

```
@returns un valor positivo que…
```

Tips documentación

7

Indiquen explícitamente todos los retornos ‘especiales’

`-1 cuando...`

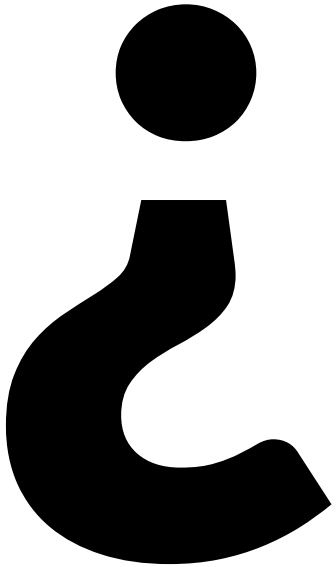
`0 cuando esta todo bien`

Ejemplo de documentación completa

```
/**
 * Operación de suma de dos operandos entre valores enteros.
 *
 * @param izquierdo es el primer operando
 * @param derecho es el operando que será sumado al izquierdo
 * @return la suma de ambos parámetros
 */
int suma(int izquierdo, int derecho)
```

Todas las funciones con documentación completa.


¿Preguntas?



Cómo refinar la documentación



Contratos

primer round

UNRN Universidad Nacional de Río Negro

¿Qué es un contrato?

Historia y Origen

Atributos

Precondición

Son condiciones que deben ser verdaderas antes de que una función sea ejecutada.

Ejemplo - división

```
// divisor no puede ser cero
int division_entera(int dividendo, int divisor)
```

Postcondición

Son condiciones que deben ser verdaderas después de que la función ha sido ejecutada.

Ejemplo - división

```
// el resultado es dividendo/divisor
// siempre y cuando divisor no sea cero
int division_entera(int dividendo, int divisor)
```

Invariante

Son condiciones que deben ser verdaderas antes y después de la ejecución de una función y a lo largo de toda la vida del programa.

Ejemplo - Lazo de suma en un arreglo

En un bucle que suma los elementos de un arreglo, la suma acumulada siempre sea igual a la suma de los elementos procesados hasta ese punto.

Lo revisamos para ver que entendieron de la consigna

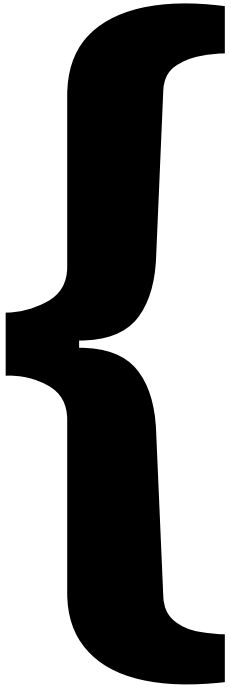
{ ejemplos }

Descripción de la función - ejemplo 1

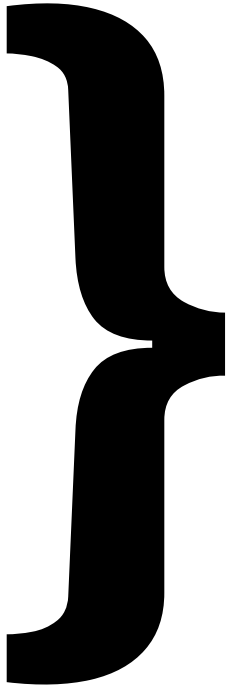
```
/*
 * Toda función debe tener un comentario que
 * describa que es lo que hace.
 * Indicando que rol cumple y hace cada argumento.
 * Esta función se encarga de sumar dos numeros enteros
 * @param izquierdo es el primer termino de la suma
 * @param derecho es el segundo termino a sumar
 * @returns la suma de ambos términos
 * PRE-CONDICION: numero enteros en el rango de +/-2^8.
 * POST-CONDICION: numero entero que es suma de los
 *                          términos de entrada. La salida tendrá
 *                          un rango entre +/- 2^9
 */
int suma(int izquierdo, int derecho);
```

Descripción de la función - ejemplo 2

```
/*
 * La descripción de la función.
 * ¿Cual es su objetivo?
 * @param termino1 es el primer término a operar
 *                 debe ser positivo y menor a 3000 (PRE)
 * @param termino2 es el segundo término a operar
 *                 debe ser negativo y mayor a 3000 (PRE)
 * @returns la operación de ambos términos
 *      el signo del resultado será el del número mayor
 *      el resultado no puede ser mayor a +/-6000 (POS)
 */
int operar_terminos(int termino1, int termino2);
```



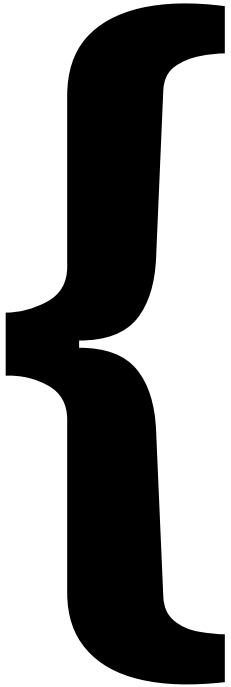
No hay un estilo correcto/incorrecto



Tiene que estar

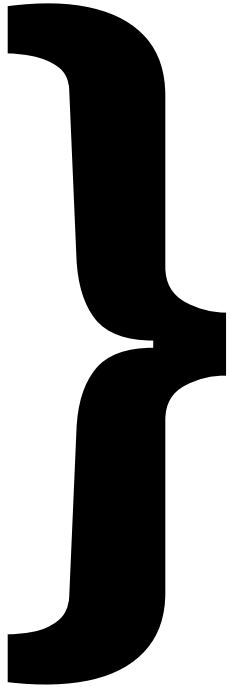
Por otro lado

UNRN Universidad Nacional de Río Negro



En la pre/post condición, indicar el tipo no es tan importante

Esto ya está dado por el tipo del argumento/retorno



Céntrense en las características que tiene el valor para probar la función

Rango de valores

¿Que valores son válidos para la función?

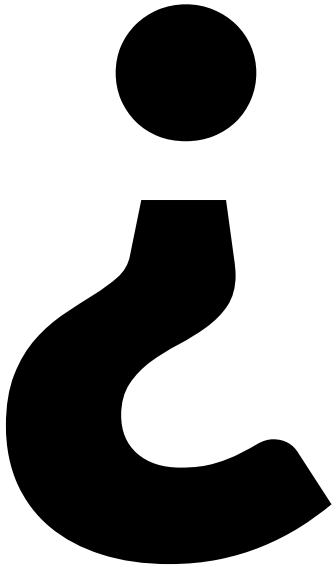
de entrada y de salida

```
El divisor no puede ser cero
El factorial no puede ser negativo
```

Valores especiales

Si usan algún valor en particular con un significado diferente al que pueda ser

```
'c' para celcius
Retorno -1 cuando no puedo calcular
```



Y las invariantes



{ Por ahora no son importantes }

Y todo esto...

¿Por qué?

¡Son una invitación a pasarles esos valores!

Hace más fácil asegurarnos de que hace lo que supuestamente debe

¡pruebas!!

Hasta la próxima

UNRN Universidad Nacional de **Río Negro**

Backup

UNRN Universidad Nacional de **Río Negro**

Más sobre documentación

Aquellas funciones que terminan describiendo con un “y”

probablemente sean *dos* funciones

Una función que probablemente sean dos

Esta función se encarga de algo y otra cosa

{ Durante el cuatrimestre vamos a ir viendo ejemplos }

Siempre que sea posible*, una responsabilidad por función.

Valores compartidos

UNRN Universidad Nacional de Río Negro

Un valor `static` sigue estando en el alcance de la función

```
int incremento(int numero)
{
    static int veces;
    veces = veces + 1;
    printf("llamamos %d veces \n", veces);
    return numero + 1;
}
```

¡Un uso piola!

```
{
    static int veces;
    veces = veces + 1;
    printf("*** DEBUG - se llamo %d veces \n", veces);
}
```

Como un bloque aislado dentro de una función.

Los usos son limitados


¿Preguntas?